Fase 5

MUITOS DADOS, UMA SÓ CAIXA: VETORES E MATRIZES



APRESENTADO POR:
LAUREN MARÇULO

Start



Imagine que precisamos guardar as notas de 30 alunos de uma turma e passá-las para uma função que calcula a média.

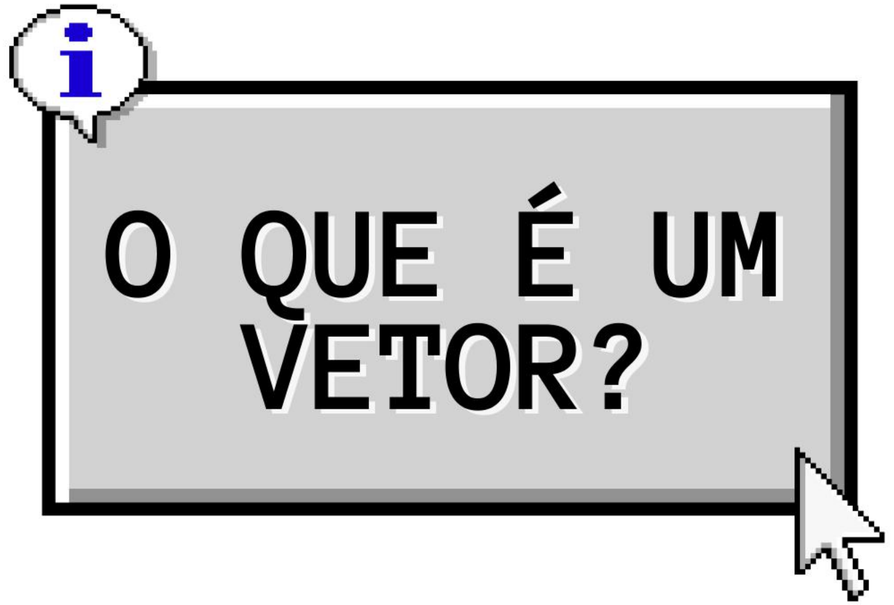
Sem vetores, teríamos que criar 30 variáveis (nota1, nota2, ..., nota30) e 30 parâmetros na função.

Os **vetores** e **matrizes** resolvem este problema permitindo agrupar múltiplos dados do mesmo tipo sob um único nome

Vetor ou array unidimensional é uma estrutura de dados que armazena uma sequência cotínua de elementos do mesmo tipo na memória e cada elemento é acessado através de um índice.

Declaração em C:

tipo nome_vetor[tamanho]



```
float notas[5];   // reserva espaço para 5 números reais
```

Estrutura na Memoria:

Índice:	[0]	[1]	[2]	[3]	[4]
Valor:	7.5	8.0	6.5	9.0	10.0

⚠ Atenção ao Índice: A contagem do índice sempre começa no 0. Um vetor de tamanho 5 possui posições válidas de 0 a 4.



ACESSANDO E GUARDANDO VALORES NO VETOR

Para guardar ou ler um valor em uma gaveta específica, colocamos o número do índice entre colchetes [].

Guardando valores

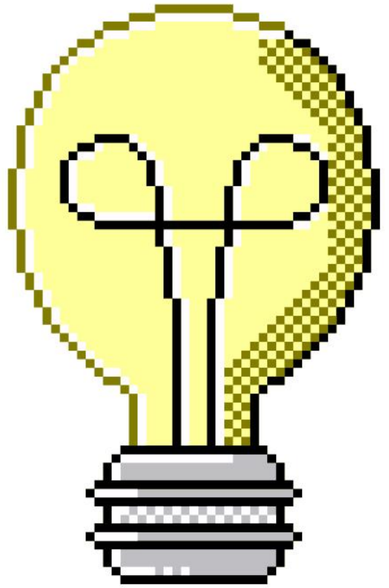
```
idade[0] = 18;
idade[1] = 25;
idade[2] = 30;
```

Lendo o valor guardado

```
printf("A primeira idade é: %d\n", idade[0]);  // Exibe 18
```

UNINDO VETORES, LAÇOS E FUNÇÕES

A maior vantagem do vetor é que o laço for é perfeito para percorrer todos os índices de uma vez!
Também podemos enviar o vetor inteiro para uma função processar.



```
// Procedimento para mostrar todos os elementos do vetor (Fase 4)

void exibirVetor(int vet[], int tamanho) {
     for (int i = 0; i < tamanho; i++) {
     // O 'i' vai de 0 ate tamanho-1
           printf("Posicao %d: %d\n", i, vet[i]);
      }
}
```

VAMOS PRATICAR VETORES!
MISSÃO 1: O TESOURO DOS CINCO BAÚS

Um grupo de aventureiros encontrou 5 baús de ouro escondidos numa masmorra, numerados de [0] a [4]. Cada baú guarda uma quantidade diferente de moedas.

>> SUA MISSÃO:

1. **Crie o vetor: int baus[5] = {15, 40, 8, 25, 12};**
2. **Crie uma variável acumuladora: int total = 0;**
3. **Use um laço for (int i = 0; i < 5; i++) para somar!**
4. **Exiba o ouro total conquistado pela equipe.**

ESBOÇO DA SOLUÇÃO EM C

```c
int baus[5] = {15, 40, 8, 25, 12};
int total = 0;

for (int i = 0; i < 5; i++) {
    total += baus[i]; // Acumula o valor
}
printf("Ouro total: %d moedas!\n", total);
```



DICA PEDAGÓGICA TRY, CATCH & LEARN:
Lembre-se: em vetores de 5 posições, o laço SEMPRE vai de 0 até tamanho-1 (índice 4)!

VAMOS PRATICAR VETORES!
MISSÃO 2: O RECORDE DO CAMPEONATO

Cinco pilotos disputaram a grande final da UTFPR.
As pontuações finais foram guardadas em um vetor.
O sistema precisa descobrir qual foi a maior nota!

>> SUA MISSÃO:

1. **Vetor de pontos: int pts[5] = {42, 88, 75, 95, 60};**
2. **Assuma inicialmente: int recorde = pts[0];**
3. **Percorra o vetor comparando: se pts[i] > recorde, atualize o valor do recorde!**
4. **Mostre o recorde e o troféu do campeão na tela.**

ESBOÇO DA SOLUÇÃO EM C

```c
int pts[5] = {42, 88, 75, 95, 60};
int recorde = pts[0]; // Começa com o 1º

for (int i = 1; i < 5; i++) {
    if (pts[i] > recorde) {
        recorde = pts[i]; // Novo campeão!
    }
}
printf("Maior pontuacao: %d pts!\n", recorde);
```



DICA PEDAGÓGICA TRY, CATCH & LEARN:
Este é o clássico algoritmo de 'busca de maior valor' — base de muitos sistemas reais!

O QUE É UMA MATRIZ?

Matriz ou array bidimensional é uma estrutura de dados que armazena uma tabela com linhas e colunas.

Declaração em C:

tipo nome_matriz[qunt. linhas][qunt. colunas]

```
int tabela[3][3];   //uma tabela de 3 linhas e 3 colunas
```

Representação visual (Grid):

	Coluna 0	Coluna 1	Coluna 2
Linha 0:	tab[0][0]	tab[0][1]	tab[0][2]
Linha 1:	tab[1][0]	tab[1][1]	tab[1][2]
Linha 2:	tab[2][0]	tab[2][1]	tab[2][2]

PERCORRENDO UMA MATRIZ COM LAÇOS ANINHADOS

Para andar por uma tabela inteira, usamos dois laços for encadeados:
1.O laço de fora escolhe a linha.
2.O laço de dentro percorre todas as colunas daquela linha.

```
int main() {
     // Criando e preenchendo uma matriz 2x3 (2 linhas, 3 colunas)
     int matriz[2][3] = {
          {1, 2, 3}, // Linha 0
          {4, 5, 6}  // Linha 1
      };
      printf("--- Exibindo a Matriz em formato de tabela ---\n");
      for (int i = 0; i < 2; i++) { //‘i' controla as Linhas
            for (int j = 0; j < 3; j++) { //'j' controla as Colunas
                  printf("%d ", matriz[i][j]);
             }
             printf("\n"); //Pula uma linha ao terminar de imprimir uma linha inteira
}
```

VAMOS PRATICAR MATRIZES!
MISSÃO 3: O TABULEIRO MÁGICO 2x2

Em um mini-RPG de tabuleiro, a arena elemental tem
2 linhas e 2 colunas guardando o poder de gemas.
Precisamos somar a energia total da arena mágica!

>> SUA MISSÃO:

1. **Declare: int arena[2][2] = { {15, 25}, {10, 30} };**
2. **Crie uma variável: int energia_total = 0;**
3. **Use dois laços for aninhados (linhas i e colunas j);**
4. **Imprima a matriz em grid e mostre a energia total.**

ESBOÇO DA SOLUÇÃO EM C

```c
int arena[2][2] = { {15, 25}, {10, 30} };
int energia = 0;
for (int i = 0; i < 2; i++) { // Linhas
    for (int j = 0; j < 2; j++) { // Colunas
        printf("%d ", arena[i][j]);
        energia += arena[i][j];
    }
    printf("\n"); // Pula linha ao fim
}
printf("Energia total: %d\n", energia);
```



DICA PEDAGÓGICA TRY, CATCH & LEARN:
Matrizes 2D sempre combinam 2 laços for: o de fora escolhe a Linha, o de dentro a Coluna!

VAMOS PRATICAR MATRIZES!
MISSÃO 4: O MAPA DA SALA DE CINEMA

Uma sala de cinema retrô possui 2 fileiras com 3 assentos cada. O valor 1 significa poltrona ocupada, e o valor 0 significa poltrona vazia / livre.

>> SUA MISSÃO:

1. **Mapa da sala: int cinema[2][3] = {{1,0,1}, {0,1,1}};**
2. **Crie um contador de espectadores: int ocupados = 0;**
3. **Percorra a matriz inteira com dois laços for;**
4. **Se cinema[i][j] == 1, incremente seu contador!**

ESBOÇO DA SOLUÇÃO EM C

```c
int cinema[2][3] = { {1, 0, 1}, {0, 1, 1} };
int ocupados = 0;
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        if (cinema[i][j] == 1) {
            ocupados++; // Mais um cliente
        }
    }
}
printf("Ingressos vendidos: %d\n", ocupados);
```



DICA PEDAGÓGICA TRY, CATCH & LEARN:
Contadores condicionais dentro de matrizes são a base de jogos como Batalha Naval e Xadrez!



PENSANDO COMO UM PROGRAMADOR

Antes de fazer o código, faça essas 3 perguntas:

1 - Quantos dados e de qual tipo preciso guardar?

2 - Quais são os índices válidos?

3 - Qual estrutura é a ideal?

O QUE VOCÊ APRENDEU ATÉ AQUI!



Ao finalizar esta fase, você aprendeu a:

- Organizar e armazenar conjuntos de dados sob um único nome.
- Usar vetores para listas de dados e matrizes para tabelas.
- Acessar cada posição com precisão usando os índices [i] e [i][j].
- Integrar vetores com laços de repetição (Fase 3) e funções (Fase 4).

OBRIGADO(A) PELA ATENÇÃO

skip stop CONTINUE

